

SKILL WEEK-1

Task-1:

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
suresh@DESKTOP-QC18C9M: ~/OSSP
The following NEW packages will be installed:
tree
0 upgraded, 1 newly installed, 0 to remove and 240 not upgraded.
Need to get 47.4 kB of archives.
After this operation, 111 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 tree amd64 2.1.1-2ubuntu3.24.04.2 [47.4 kB]
Fetched 47.4 kB in 1s (50.6 kB/s)
Selecting previously unselected package tree.
(Reading database ... 46671 files and directories currently installed.)
Preparing to unpack .../tree_2.1.1-2ubuntu3.24.04.2_amd64.deb ...
Unpacking tree (2.1.1-2ubuntu3.24.04.2) ...
Setting up tree (2.1.1-2ubuntu3.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...
suresh@DESKTOP-QC18C9M:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── Practical
├── Practical
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executer.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── builtin.h
│   ├── executer.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executer.c
    └── test_parser.c

11 directories, 19 files
```

Task-2:

To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1:

Using a manual command understand the fork() and exec() system calls

Task-2:

Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork(); // Create a new process
```

```
if(pid < 0) {  
    printf("Fork failed!\n");  
}  
else if(pid == 0) {  
    // Child Process  
  
    printf("This is the Child Process.\n");  
    printf("Child PID = %d\n", getpid());  
}  
else {  
    // Parent Process  
  
    printf("This is the Parent Process.\n");  
    printf("Parent PID = %d\n", getpid());  
}  
}
```

```
drwxr-xr-x 2 suresh suresh 4096 Sep  2 03:20 src
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:19 tests

Child process completed.
suresh@DESKTOP-QCI8C9M:~/OSSP$ ls
ls: command not found
suresh@DESKTOP-QCI8C9M:~/OSSP$ pwd
/home/suresh/OSSP
suresh@DESKTOP-QCI8C9M:~/OSSP$ date
Wed Sep  2 03:31:26 UTC 2026
suresh@DESKTOP-QCI8C9M:~/OSSP$ nano.process.c
nano.process.c: command not found
suresh@DESKTOP-QCI8C9M:~/OSSP$ gcc process.c -o process
suresh@DESKTOP-QCI8C9M:~/OSSP$ ./process
Parent Process Started
Parent PID = 970

Parent Process
Parent PID = 970

Child PID = 971
Child Process Created
Child PID = 971
Parent PID = 970
Executing 'ls' command...
total 60
-rw-r--r-- 1 suresh suresh    0 Sep  2 03:22 LICENSE
-rw-r--r-- 1 suresh suresh    0 Sep  2 03:22 Makefile
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:41 Practical
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:40 Prctical
-rw-r--r-- 1 suresh suresh    0 Sep  2 03:22 README.md
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:19 assets
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:13 bin
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:18 docs
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:16 include
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:13 obj
-rwxr-xr-x 1 suresh suresh 16264 Sep  2 03:31 process
-rw-r--r-- 1 suresh suresh   997 Sep  2 03:30 process.c
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:19 scripts
drwxr-xr-x 2 suresh suresh 4096 Sep  2 03:20 src
drwxr-xr-x 2 suresh suresh 4096 Aug 14 05:19 tests

Child process completed.
```

NAME

fork - create a child process

LIBRARY

Standard C library (`libc`, `-lc`)

SYNOPSIS

```
#include <unistd.h>
```

```
pid_t fork(void);
```

DESCRIPTION

`fork()` creates a new process by duplicating the calling process. The new process is referred to as the child process. The calling process is referred to as the parent process.

The child process and the parent process run in separate memory spaces. At the time of `fork()` both memory spaces have the same content. Memory writes, file mappings (`mmap(2)`), and unmappings (`munmap(2)`) performed by one of the processes do not affect the other.

The child process is an exact duplicate of the parent process except for the following points:

- The child has its own unique process ID, and this PID does not match the ID of any existing process group (`setpgid(2)`) or session.
- The child's parent process ID is the same as the parent's process ID.
- The child does not inherit its parent's memory locks